_ TELECOM ESCUELA TÉCNICA VLC SUPERIOR DE INGENIERÍA DE TELECOMUNICACIÓN

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

GRADO EN TECNOLOGÍA DIGITAL Y MULTIMEDIA

ASIGNATURA: SISTEMAS EMBEBIDOS

MEMORIA TRABAJO FINAL: GENERADOR MUSICAL

CURSO 2023/2024

PAULA OTERO PAÑOS

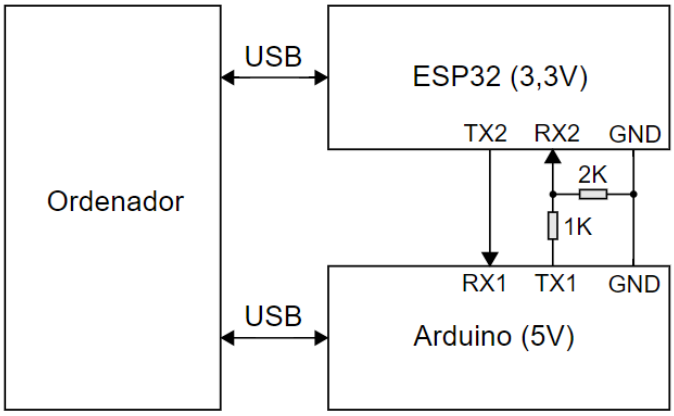
ANNA REIG BORRÁS

Índice

1. INTRODUCCIÓN 1
2. OBJETIVOS 2
3. DESARROLLO 3
3.1. Melodías (melodies.ino) 3
3.2. WiFi (wifi.ino) 11
4. CONCLUSIONES 17
5. ANEXO 18

1. INTRODUCCIÓN

La programación de sistemas en entornos embebidos, como Arduino, ofrece una plataforma versátil para la implementación de diversas aplicaciones prácticas. Estos sistemas, integrados en el hardware que controla su funcionamiento, son fundamentales en la vida moderna, abarcando desde electrodomésticos hasta sistemas industriales complejos. En este documento se aborda el desarrollo de una aplicación práctica que involucra la creación y reproducción de melodías, utilizando una configuración que integra el Arduino Mega 2560 y la tarjeta ESP32-DevKitV1 conectados a través de UART.



El objetivo principal de esta práctica es la programación de comunicaciones cableadas e inalámbricas en C con Arduino. Para soportar la funcionalidad, se utiliza la tarjeta ESP32, que actúa como intermediario para la transmisión de melodías creadas por el usuario. El usuario puede seleccionar y gestionar estas melodías a través de una página web, aprovechando la comunicación WiFi. La ESP32 envía la melodía seleccionada al Arduino Mega 2560, que la reproduce mediante un zumbador. Además, se incorporan un visualizador LCD y un codificador rotatorio para mejorar la interacción y visualización de la información. La página HTML se gestiona a través de SPIFFS, permitiendo modificar el contenido HTML según sea necesario.

Este proyecto aborda de manera integral las diferentes etapas del desarrollo, desde la planificación y diseño hasta las implementación y pruebas. Se exploran las decisiones de diseño, las técnicas de programación empleadas y las soluciones implementadas para superar los desafíos encontrados. El proceso de desarrollo se ilustra mediante pseudocódigo, que proporciona una visión general de la estructura del código y de las operaciones realizadas en cada etapa. Además, se incluye esquemas de montaje y diagramas que ayudan a visualizar la configuración física de la creación y reproducción de melodías.

El desarrollo del sistema incluye la configuración de la tarjeta ESP32 para la comunicación con el Arduino Mega 2560, permitiendo al usuario seleccionar y reproducir melodías a través de una interfaz web. Las melodías son predeterminadas y creadas por el usuario, demostrando así la versatilidad de la ESP32 y la potencia del entorno de programación Arduino IDE. La implementación de estas funcionalidades no solo pone en evidencia las capacidades de los dispositivos utilizados, sino también el esfuerzo y la dedicación invertidos en el proyecto. A lo largo de las clases y cuatro sesiones adicionales en el laboratorio, se avanzó paso a paso, optimizando los códigos mediante la creación de múltiples funciones para mejorar la eficiencia y la claridad del programa.

2. OBJETIVOS

El desarrollo de la funcionalidad de los periféricos implementados en el proyecto, así como la comunicación efectiva entre el Arduino Mega 2560 y la tarjeta ESP32, se establecen como los principales objetivos del trabajo. Para lograr este propósito, se establecen los siguientes objetivos específicos:

- Implementar la funcionalidad del visualizador LCD, permitiendo una visualización clara, como el total de melodías creadas.
- Desarrollar la lógica del control del codificador rotatorio, facilitando la interacción del usuario para la creación de melodías.
- Integrar la capacidad del zumbador para reproducir las melodías seleccionadas por el usuario, asegurando una reproducción precisa y clara de las mismas.
- Establecer una comunicación efectiva entre el Arduino Mega 2560 y la tarjeta ESP32 a través de UART, permitiendo la transmisión de datos entre ambos dispositivos de manera fiable
- Implementar la gestión de la página HTML a través de SPIFFS, posibilitando la modificación del contenido HTML según las necesidades del proyecto y la interacción del usuario con la interfaz web.
- Documentar adecuadamente el proceso de desarrollo, incluyendo esquemas de montaje, algoritmos utilizados y vídeos de funcionamiento de las tareas destacadas.

3. DESARROLLO

En esta sección, se explora el proceso de desarrollo del código para la funcionalidad de los periféricos en sistemas embebidos, utilizando Arduino Mega 2560 y la ESP32. Desde el diseño de la interfaz de usuario hasta la integración de los dispositivos y la comunicación entre ellos, se detallará el enfoque utilizado para alcanzar los objetivos establecidos en el proyecto. A lo largo de este apartado, se examinan las decisiones de diseño, las técnicas de programación empleadas y las soluciones implementadas para superar los desafíos encontrados durante el desarrollo.

3.1. Melodías (melodies.ino)

Inicialmente, se planificó el algoritmo del codificador rotativo. Una vez se supo como gestionarlo se empezó con la elaboración y desarrollo del código. A lo largo, fueron surgiendo algunos problemas, como la visualización del LCD.

A continuación, se describen las funciones principales y los aspectos clave del código, destacando las modificaciones realizadas para solucionar los problemas que surgieron.

Variables globales utilizadas:

- ‘`pinA`’, ‘`pinB`’ y ‘`pinSw`’: Representan los pines correspondientes al interruptor (Switch), la señal de datos (DT), y la señal del reloj (CLK) respectivamente.
- ‘`currentNoteIndex`’: Almacena el índice de la nota actual en la melodía en creación.
- ‘`currentMelodyIndex`’: Almacena el índice de la melodía actual en el arreglo de melodías.
- ‘`noteSelection`’: Almacena el índice de la nota seleccionada por el usuario.
- ‘melodyCount’: Almacena el número total de melodías creadas.
- ‘confirmSave’: Indica si se ha confirmado guardar la melodía actual.
- ‘lastSwitchPressTime’: Almacena el tiempo de la última pulsación del interruptor.
- ‘switchPressCount’: Almacena el número de pulsaciones del interruptor.
- Arreglos ‘noteFrequencies[]’ y ‘noteNames[]’: Almacenan las frecuencias y nombres de las notas respectivamente.

La función ‘setupPins()’ se encarga de inicializar los pines indicados para que realicen sus funciones correspondientes.

La función ‘handleEncoder()’ se encarga de manejar la lógica del codificador rotatorio y actualizar una selección de notas en función del giro detectado.

La función ‘handleSwitchPress()’ maneja la lógica asociada a la detección de pulsaciones de un interruptor, incluyendo la gestión de rebotes y la ejecución de diferentes acciones basadas en el número de pulsaciones.

En primer lugar, se detecta si el interruptor ha sido pulsado (LOW) y se asegura de que no haya rebotes utilizando una verificación de tiempo (‘debounceTime’). Si se detecta una pulsación válida, se incrementa un contador (‘switchPressCount’) y se actualiza el tiempo de la última pulsación. Cuando se detecta que el interruptor ya no está pulsado, se verifica cuántas pulsaciones se registraron y se ejecuta la acción correspondiente. Si detecta una pulsación, se añade la nota seleccionada a la melodía actual. Si detecta dos, se guarda la melodía creada. Si detecta tres, se elimina la última nota almacenada. Y si detecta cuatro, se eliminan todas las notas almacenadas y se vuelve desde cero a almacenar notas para la melodía actual. Por último, el contador de pulsaciones se reinicia para la siguiente detección.

La función ‘playMelodiesFromESP32()’ se encarga de reproducir la melodía seleccionada en la página web. Esta llama a ‘playMelody()’ para que reproduzca las notas de la melodía.

La función ‘sendMelodyToESP32()’ se encarga de enviar las melodías creadas por el usuario.

Se crearon funciones como ‘handleSwitchPress()’ y ‘confirmSaveLoop()’ para manejar la interacción con el usuario y la visualización en la pantalla LCD.

En cuanto a la visualización en el LCD, se presentaron varios problemas. Al pasar a la siguiente nota para elegir la nota para crear la melodía, cuando se pasaba por la nota ‘SOL’ y luego se iba a otra cualquiera, la ‘L’ se quedaba en la pantalla. Para solucionar esto, se añadió un ‘lcd.print(“ “);’ antes de actualizar la nota para limpiar el área y evitar caracteres residuales. Además, los corchetes que indicaban la opción seleccionada se quedaban en la pantalla de forma incorrecta, mostrando ‘[Si] [No’ en lugar de ‘[Si] No’ o ‘Si [No]’. Se aplicó el mismo enfoque de limpieza con espacios en blanco antes de mostrar los corchetes, asegurando que la pantalla se actualizara correctamente sin dejar caracteres residuales. También, al borrar la última nota almacenada cuando ‘currentNoteIndex=10’, el dígito ‘0’ se quedaba en la pantalla. Se implementó una limpieza del área de visualización de las notas antes de actualizar la pantalla, garantizando que los dígitos sobrantes fueran eliminados. Asimismo, al borrar todas las notas almacenadas cuando ‘currentNoteIndex>=10’, el segundo dígito se quedaba en la pantalla. Se extendió la limpieza del área de visualización para manejar correctamente los dígitos de dos cifras, asegurando que al borrar todas las notas no quedaran caracteres residuales.

En cuanto al envío del nombre de la melodía creada al ESP32, no se enviaba ningún nombre de la melodía, solo se enviaba un carácter vacío. La solución fue redefinir la variable que almacena el nombre de la melodía creada justo después de la asignación del nombre. Anteriormente, esta definición estaba después de la confirmación de crear otra melodía, lo que causaba que '`currentMelodyIndex`' se incrementara antes de tiempo. Al mover la definición, se aseguró que el nombre correcto se enviara al ESP32.

Respecto a la gestión del codificador rotativo, las pulsaciones eran confundidas, ya que el usuario debía realizarlas muy rápido, lo que resultaba en errores en la detección. Para solucionar esto, se añadieron retardos ('`delay`') para evitar los rebotes del codificador rotatorio. Esto permitió que el sistema tuviera tiempo suficiente para estabilizarse entre las pulsaciones, mejorando la precisión en la detección de las acciones del usuario.

Finalmente, en cuanto a la comunicación con el ESP32, al seleccionar cualquier melodía desde la página, esta no llegaba al Arduino, por lo que no podía ser reproducida. La solución fue inicializar correctamente otro puerto serial utilizando '`Serial1.begin(115200);`'. Esto permitió la comunicación adecuada entre el ESP32 y el Arduino, asegurando que las melodías seleccionadas en la página web llegaran correctamente al Arduino para su reproducción. Este mismo problema pasaba al enviar la melodía creada a la tarjeta ESP32.

Estas modificaciones y soluciones permitieron corregir los problemas encontrados y garantizar el correcto funcionamiento del sistema, mejorando tanto la experiencia del usuario como la fiabilidad de la comunicación y la visualización.

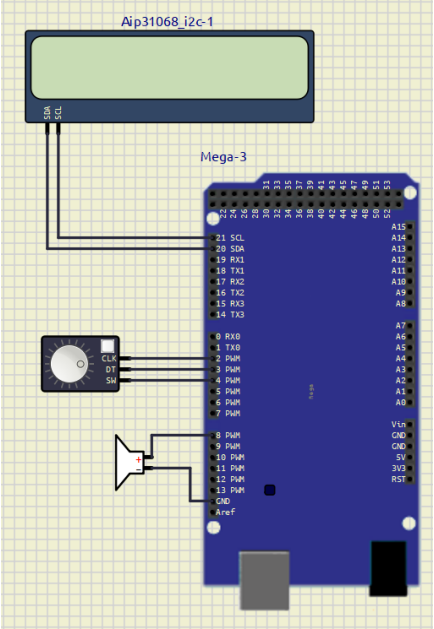


Figura 1. Esquema de montaje de Melodies

Pseudocódigo:

```
// Incluir librerías y definir constantes
Incluir la librería "rgb_lcd.h"
Incluir el archivo de tonos "pitches.h"

Definir la longitud máxima del nombre de una melodía como 15
Definir la cantidad máxima de notas por melodía como 30
Definir la cantidad máxima de melodías como 10

// Declarar variables para los pines de conexión
Entero pinA = 2
Entero pinB = 3
Entero pinSw = 4
Entero pinZ = 8

// Variables de control para la selección y gestión de melodías
Entero currentNoteIndex = 0
Entero currentMelodyIndex = 0
Entero noteSelection = 0
Entero melodyCount = 0
Booleano inCreationMode = Verdadero
Booleano confirmSave = Falso
Entero lastSwitchPressTime = 0
Entero switchPressCount = 0
Entero debounceTime = 200

// Arrays de frecuencias de notas y nombres de notas
Array de enteros noteFrequencies[] = { NOTE_C4, NOTE_D4, NOTE_E4,
NOTE_F4, NOTE_G4, NOTE_A4, NOTE_B4, NOTE_C5 }
Array de cadenas de caracteres noteNames[] = { "DO", "RE", "MI", "FA",
"SOL", "LA", "SI", "DO" }

// Declarar la estructura para representar una nota
Estructura Nota:
    Entero frequency
    Entero duration

// Declarar la estructura para representar una melodía
Estructura Melodia:
    Caracter name[MELODY_NAME_LEN + 1]
    Nota notes[MELODY_NOTES]
    Entero noteCount

// Array de melodías para almacenar las creadas
Array de Melodia melodies[MAX_MELODIES]

// Melodías predefinidas
Definir la constante Array de Melodia predefinedMelodies[]
```

```
    Primera Melodía "Star Wars":
        Nombre: "Melodia A"
        Notas: { {NOTE_G4, 500}, {NOTE_G4, 500}, {NOTE_G4, 500},
{NOTE_E4, 375}, {NOTE_B4, 125}, {NOTE_G4, 500}, {NOTE_E4, 375}, {NOTE_B4,
125}, {NOTE_G4, 1000}, {-1, 0} }
    Segunda Melodía "Harry Potter":
        Nombre: "Melodia B"
        Notas: { {NOTE_E5, 300}, {NOTE_G5, 300}, {NOTE_FS5, 300},
{NOTE_E5, 300}, {NOTE_B5, 300}, {NOTE_A5, 300}, {NOTE_FS5, 300},
{NOTE_D5, 300}, {NOTE_B5, 300}, {NOTE_A5, 300}, {NOTE_FS5, 300},
{NOTE_E5, 300}, {NOTE_G5, 300}, {NOTE_FS5, 300}, {NOTE_D5, 300},
{NOTE_A5, 300}, {NOTE_FS5, 300}, {NOTE_E5, 600}, {-1, 0} }
    Tercera Melodía "Indiana Jones":
        Nombre: "Melodia C"
        Notas: { {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_G4, 600},
{NOTE_C4, 300}, {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_D4, 300}, {NOTE_E4,
600}, {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_G4, 600}, {NOTE_C4, 300},
{NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_D4, 300}, {NOTE_E4, 600}, {-1, 0} }
    Cuarta Melodía "Piratas del Caribe":
        Nombre: "Melodia D"
        Notas: { {NOTE_E4, 300}, {NOTE_G4, 300}, {NOTE_A4, 300},
{NOTE_B4, 300}, {NOTE_C5, 300}, {NOTE_D5, 300}, {NOTE_E5, 300}, {NOTE_G5,
600}, {NOTE_E5, 300}, {NOTE_D5, 300}, {NOTE_C5, 300}, {NOTE_B4, 300},
{NOTE_A4, 300}, {NOTE_G4, 300}, {NOTE_E4, 300}, {NOTE_D4, 600}, {NOTE_E4,
300}, {NOTE_G4, 300}, {NOTE_A4, 300}, {NOTE_B4, 300}, {NOTE_C5, 300},
{NOTE_D5, 300}, {NOTE_E5, 300}, {NOTE_G5, 600}, {-1, 0} }

// Configuración inicial
Función setup():
    Iniciar la comunicación serial a 115200 baudios
    Iniciar la comunicación serial1 a 115200 baudios
    setupPins()
    setupLCD()

// Configuración de pines
Función setupPins():
    Configurar pinA como entrada
    Configurar pinB como entrada
    Configurar pinSw como entrada con resistencia pull-up
    Configurar pinZ como salida

// Configuración de la pantalla LCD
Función setupLCD():
    Iniciar la pantalla LCD con 16 columnas y 2 filas
    Establecer el color RGB de la pantalla como (200, 200, 200)
    Mostrar "Creador de" en la primera fila de la pantalla
    Mostrar "melodias" en la segunda fila de la pantalla
    Esperar 2000 milisegundos
    Limpiar la pantalla
```

```
    Mostrar "Introduce 10 // notas" en la primera fila de la pantalla
    Mostrar "Nota: DO" en la segunda fila de la pantalla
    Esperar 2000 milisegundos
    Limpiar la pantalla
    Mostrar "Nota: DO" en la segunda fila de la pantalla

// Manejar el codificador rotativo
Función handleEncoder():
    Establecer el estado anterior del pin A del codificador rotativo
    Leer el estado actual del pin A del codificador rotativo
    Si el estado actual de pin A es diferente al estado anterior de pin A
y el estado actual de pin A es ALTO:
        Si el pin B del codificador rotativo está en BAJO:
            Incrementar la selección de nota en 1, manteniéndola dentro
del rango de 0 a 7
        Sino:
            Decrementar la selección de nota en 1, manteniéndola dentro
del rango de 0 a 7
        Borrar el espacio de visualización de la nota seleccionada en la
pantalla LCD
        Actualizar el espacio de visualización de la nota seleccionada en
la pantalla LCD

// Manejar la pulsación del interruptor
Función handleSwitchPress():
    Verificar si el interruptor está presionado
    Si el interruptor está presionado y ha pasado más tiempo que el
tiempo de rebote desde la última pulsación:
        Incrementar el contador de pulsaciones del interruptor
        Establecer el tiempo de la última pulsación del interruptor
        Esperar 100 milisegundos

    Si el interruptor no está presionado y ha pasado más tiempo que el
tiempo de rebote desde la última pulsación:
        Si el contador de pulsaciones del interruptor es 1:
            Añadir una nota a la melodía
        Sino, si el contador de pulsaciones del interruptor es 2:
            Guardar la melodía
        Sino, si el contador de pulsaciones del interruptor es 3:
            Eliminar la última nota de la melodía
        Sino, si el contador de pulsaciones del interruptor es 4:
            Eliminar todas las notas de la melodía
        Reiniciar el contador de pulsaciones del interruptor

// Añadir una nota a la melodía actual
Función addNote():
    Añadir la frecuencia de la nota seleccionada con una duración de 250
milisegundos a las notas de la melodía actual
    Incrementar el índice de la nota actual
```

```
    Actualizar el espacio de visualización del número de notas en la
pantalla LCD

// Guardar la melodía actual
Función saveMelody():
    Establecer el contador de notas de la melodía actual
    Incrementar el contador de melodías
    Generar un nombre para la melodía actual
    Mostrar "Melodia guardada" en la pantalla LCD
    Esperar 2000 milisegundos
    Limpiar la pantalla
    Mostrar "Crear otra?" en la primera fila de la pantalla LCD
    Mostrar "[Si] No" en la segunda fila de la pantalla LCD
    Establecer confirmSave en VERDADERO
    Entrar en un bucle para confirmar si se quiere crear otra melodía

// Bucle para confirmar la acción de guardar la melodía
Función confirmSaveLoop():
    Establecer el estado anterior del pin A del codificador rotativo
    Mientras (Verdadero):
        Leer el estado actual del pin A del codificador rotativo
        Si el estado actual de pin A es diferente al estado anterior de
pin A y el estado actual de pin A es ALTO:
            Si el pin B del codificador rotativo está en BAJO:
                Cambiar el estado de confirmSave
            Sino:
                Cambiar el estado de confirmSave
            Borrar el espacio de visualización de la confirmación en la
pantalla LCD
            Actualizar el espacio de visualización de la confirmación en
la pantalla LCD
        Actualizar el estado anterior de pin A del codificador rotativo
        Si el interruptor está presionado:
            Esperar 200 milisegundos
            Si confirmSave es VERDADERO:
                Establecer el índice de la nota actual a 0
                Establecer el índice de la melodía actual al contador de
melodías
                Limpiar la pantalla
                Mostrar "Nota: DO" en la segunda fila de la pantalla LCD
            Sino:
                Limpiar la pantalla
                Mostrar "Total melodias:" en la primera fila de la
pantalla LCD
                Mostrar el contador de melodías en la segunda fila de la
pantalla LCD
                Mientras (Verdadero):
                    Continuar // Mantener el mensaje
            Romper el bucle
```

```
// Eliminar la última nota de la melodía actual
Función deleteLastNote():
    Si el índice de la nota actual es mayor que 0:
        Decrementar el índice de la nota actual
        Actualizar el espacio de visualización del número de notas en la
pantalla LCD

// Eliminar todas las notas de la melodía actual
Función deleteAllNotes():
    Establecer el índice de la nota actual a 0
    Actualizar el espacio de visualización del número de notas en la
pantalla LCD

// Reproducir una melodía con una frecuencia y duración dadas
Función playMelody(Entero frecuencia, Entero duración):
    Si la frecuencia es igual a 0:
        Detener el tono en el pinZ
    Sino:
        Generar un tono en el pinZ con la frecuencia y duración dadas
    Esperar la duración de la nota

// Reproducir melodías desde el ESP32
Función playMelodiesFromESP32(Cadena melodyName):
    Para cada Melodía en predefinedMelodies:
        Si el nombre de la melodía actual es igual a melodyName:
            Para cada Nota en la melodía actual:
                Reproducir la nota con la frecuencia y duración
especificadas
            Retornar
    Para cada Melodía en melodies:
        Si el nombre de la melodía actual es igual a melodyName:
            Para cada Nota en la melodía actual:
                Reproducir la nota con la frecuencia y duración
especificadas
            Retornar
    Mostrar "Error: Melodía no encontrada" en la consola serial

// Enviar la melodía al ESP32
Función sendMelodyToESP32(Cadena melodyName):
    Enviar "M:" seguido del nombre de la melodía al ESP32 a través de la
comunicación serial1
    Mostrar el mensaje "Sending melody: [nombre de la melodía]" en la
consola serial
    Esperar 100 milisegundos

// Bucle principal
Repetir para siempre:
    Llamar a handleEncoder()
```

```
    Llamar a handleSwitchPress()
    Si hay datos disponibles en Serial1:
        Leer el mensaje recibido desde Serial1 hasta encontrar un salto
de línea
        Si el mensaje comienza con "M:":
            Extraer el nombre de la melodía del mensaje
            Reproducir la melodía correspondiente al nombre recibido
desde el ESP32 utilizando la función playMelodiesFromESP32()

// Fin del pseudocódigo.
```

3.2. WiFi (wifi.ino)

Inicialmente, el código HTML se implementó directamente en el mismo código fuente. Una vez comprobado el buen funcionamiento del programa, se procedió a implementar el sistema de archivos SPIFFS para una mejor gestión de los recursos web. Para ello, fue necesario crear particiones específicas para SPIFFS, permitiendo así subir los archivos necesarios al sistema de archivos del ESP32.

Para realizar las particiones, se creó una nueva carpeta llamada SPIFFS_data, proporcionada por el profesor. El contenido de esta carpeta incluye:

- 'deploy_spiffs', el archivo a ejecutar para crear las particiones.
- 'spiffs.bin', un archivo binario que se genera al ejecutar 'deploy_spiffs'.
- Una carpeta denominada data, que contiene los archivos que se subirán a la memoria del ESP32: el HTML de la página web, el icono de la página y una imagen.

Este código crea un punto de acceso WiFi y configura un servidor web para controlar las rutas de la página web. A continuación, se describirán las funciones y los aspectos clave del código, destacando las modificaciones realizadas para solucionar los problemas que surgieron.

Variables globales utilizadas:

- '`ssid`': Almacena el nombre del punto de acceso WiFi que el ESP32 crea.
- '`password`': Almacena la contraseña para conectarse al punto de acceso WiFi creado por el ESP32.
- '`Melody`': Define una estructura para almacenar el nombre de la melodía.
- '`MAX_MELODIES`': Define el número máximo de melodías que se pueden almacenar.
- '`createdMelodies`': Array de estructuras 'Melody' que almacena las melodías creadas.
- '`createdMelodyCount`': Contador que lleva la cuenta del número de melodías creadas almacenadas en '`createsMelodies`'.

- ‘melodyPlaying’: Variable de control que indica si una melodía está en reproducción para evitar múltiples reproducciones simultáneas.

La función ‘setup()’ se encarga de inicializar el sistema, configurando tanto la comunicación serial como la WiFi, así como el sistema de archivos SPIFFS y el servidor web. Se define el nombre del punto de acceso WiFi como ‘ESP32-AP04’ y se asigna una contraseña. Además, se configuran las rutas del servidor web para manejar diversas solicitudes y se inicia el servidor.

La función ‘loop()’ se encarga de gestionar las solicitudes de los clientes al servidor web. Cuando un cliente se conecta y envía una solicitud HTTP, el servidor maneja la solicitud correspondiente, ya sea para la página principal, el listado de melodías, la reproducción de una melodía, o la carga de archivos de imagen. También se lee la comunicación serial del puerto ‘Serial2’ para recibir y almacenar nombres de melodías creadas, asegurando que no se superen los límites predefinidos y actualizando la lista de melodías disponibles.

Las funciones ‘handleRoot()’, ‘handleIcon()’ y ‘handleImage()’ manejan solicitudes HTTP específicas para servir archivos almacenados en el sistema de archivos SPIFFS. La primera función abre y envía el archivo ‘index.html’, que contiene la página principal de la aplicación web. La segunda abre y envía el archivo ‘icono.jpg’, proporcionando el icono de la página web. Y, por último, la última abre y envía el archivo ‘image.png’, mostrando una imagen en la página web. En cada una de estas funciones, si el archivo no se encuentra, se envía un error 404 o 500 al cliente; si se encuentra, el contenido del archivo se lee y se envía al cliente con el tipo MIME apropiado, asegurando que los recursos necesarios se entreguen correctamente a través del servidor web.

La función ‘handleMelodies()’ maneja las solicitudes HTTP para proporcionar una lista de melodías disponibles. Primero, construye una cadena HTML con una lista de melodías predefinidas, numeradas de la ‘A’ a la ‘D’. Luego, añade un separador y construye una lista con las melodías creadas por el usuario, recorriendo el array ‘createdMelodies’ hasta el número de melodías creadas. Finalmente, envía esta cadena HTML al cliente con un tipo MIME de texto plano, asegurando que el cliente reciba la lista completa de melodías disponibles para su selección y reproducción.

La función ‘handlePlay()’ maneja las solicitudes HTTP para reproducir melodías desde la interfaz web. Primero, verifica si se ha proporcionado el argumento ‘melody’ y si no hay una melodía en reproducción; si alguna condición falla, responde con un error 400. Si la solicitud es válida, obtiene el nombre de la melodía, elimina los espacios en blanco, y lo envía a través de los puertos seriales ‘Serial’ y ‘Serial2’ para que el Arduino inicie la reproducción. Luego, envía una confirmación al cliente indicando que la melodía está en reproducción.

En cuanto a ejecutar el archivo para crear las particiones SPIFFS, se presentaron varios problemas. Primero, las rutas de ‘MKSPIFFS’, ‘ESPTOOL’ y ‘PARTITION_FILE’

estaban mal configuradas, ya que en nuestro portátil teníamos versiones diferentes. Tuvimos que actualizar estas rutas a las versiones correctas. Además, aparecía un mensaje de error indicando que el puerto del ESP32 no existía. Este problema surgía porque el programa Arduino IDE estaba abierto y utilizando el puerto para compilar el código WiFi. La solución fue ejecutar el archivo bat con el programa Arduino IDE cerrado, para asegurarse de que el puerto no estuviera ocupado y así poder realizar las particiones correctamente.

En cuanto con la comunicación con el Arduino, no se enviaba ni recibía el nombre de las melodías correctamente. La solución fue inicializar un nuevo puerto serial con '`Serial2.begin(115200);`'. Esto permitió establecer una comunicación adecuada entre la tarjeta ESP32 y el Arduino, asegurando que los nombres de las melodías fueran correctamente enviados y recibidos.

Finalmente, en cuanto a la reproducción de las melodías seleccionadas por el usuario, al seleccionar cualquier melodía, ya sea de las predefinidas o de las credas por el usuario, se reproducían duplicadas en el zumbador. Esto ocurría porque el nombre de la melodía seleccionada se enviaba dos veces, lo que causaba que el Arduino lo leyera y reprodujera dos veces. La solución fue definir una variable, '`melodyPlaying`', para controlar la reproducción. Se marcó como '`true`' al comenzar la reproducción, luego se introdujo un retraso para evitar la doble reproducción, y finalmente se marcó como '`false`' para indicar que la reproducción había finalizado. Esto aseguró que las melodías se reprodujeran una sola vez, evitando duplicaciones.

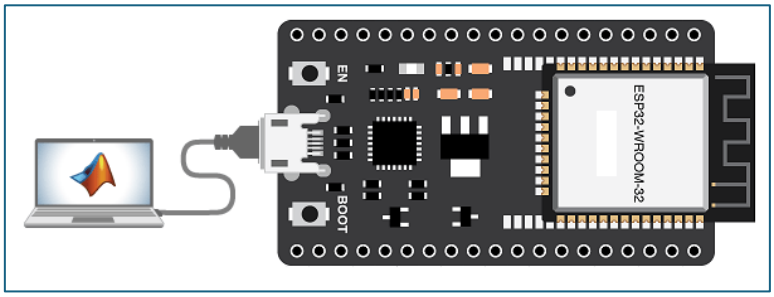


Figura 2. Esquema de montaje de WiFi

Pseudocódigo:

```
// Incluir librerías necesarias
Incluir la librería "WiFi.h"
Incluir la librería "WebServer.h"
Incluir la librería "FS.h"
Incluir la librería "SPIFFS.h"

// Definir las credenciales de la red WiFi
Constante cadena ssid = "ESP32-AP04"
```

```
Constante cadena password = "123456789"

// Crear un servidor web en el puerto 80
WebServer server(80)

// Definir la estructura para representar una melodía
Estructura Melodia:
    Cadena nombre

// Definir la cantidad máxima de melodías creadas
Definir la constante MAX_MELODIES como 10

// Declarar un array para almacenar las melodías creadas
Array de Melodia createdMelodies[MAX_MELODIES]
Entero createdMelodyCount = 0

// Variable de control para evitar múltiples reproducciones simultáneas
Booleano melodyPlaying = Falso

// Configuración inicial del sistema
Función setup():
    Iniciar la comunicación serial a 115200 baudios
    Iniciar la comunicación serial2 a 115200 baudios
    setupSPIFFS()
    setupWiFi()
    setupServer()

// Inicializar el sistema de archivos SPIFFS
Función setupSPIFFS():
    Si SPIFFS.begin(true) es Falso:
        Mostrar "Montaje de SPIFFS fallido" en la consola serial
        Retornar

// Configurar el punto de acceso WiFi
Función setupWiFi():
    Mostrar un salto de línea en la consola serial
    Mostrar "Configurando punto de acceso..." en la consola serial
    Configurar un punto de acceso WiFi con el SSID y la contraseña
definidos
    Obtener y mostrar la dirección IP del punto de acceso en la consola
serial

// Configurar las rutas y las funciones del servidor web
Función setupServer():
    Manejar la ruta raíz "/" llamando a la función handleRoot
    Manejar la ruta "/play" llamando a la función handlePlay
    Manejar la ruta "/melodies" llamando a la función handleMelodies
    Manejar la ruta "/icono.jpg" llamando a la función handleIcon
    Manejar la ruta "/image.png" llamando a la función handleImage
```

```
    Iniciar el servidor web
    Mostrar "Servidor HTTP iniciado" en la consola serial

// Manejar la solicitud de la página HTML principal
Función handleRoot():
    Abrir el archivo "index.html" desde SPIFFS
    Si no se puede abrir el archivo:
        Enviar un código de estado 500 junto con el mensaje "Error al
abrir /index.html" como respuesta al cliente
        Retornar

    Leer el contenido del archivo y almacenarlo en una cadena
    Cerrar el archivo
    Enviar un código de estado 200 junto con el contenido HTML al cliente

// Manejar la solicitud del listado de melodías
Función handleMelodies():
    Crear una cadena para almacenar el listado de melodías predefinidas y
creadas
    Para cada índice de 0 a 3:
        Agregar al listado de melodías predefinidas un enlace a la
melodía con un evento "onclick" que llama a la función "playMelody" con
el nombre de la melodía correspondiente
    Agregar una línea horizontal al listado de melodías
    Para cada melodía creada:
        Agregar al listado de melodías creadas un enlace a la melodía con
un evento "onclick" que llama a la función "playMelody" con el nombre de
la melodía correspondiente
    Enviar un código de estado 200 junto con el listado de melodías al
cliente

// Manejar la solicitud de reproducción de las melodías
Función handlePlay():
    Si no se ha especificado la melodía en la solicitud o una melodía ya
se está reproduciendo:
        Enviar un código de estado 400 junto con el mensaje "Solicitud
inválida" como respuesta al cliente
        Retornar

    Obtener el nombre de la melodía desde la solicitud
    Eliminar los espacios en blanco al principio y al final del nombre de
la melodía

    Enviar el nombre de la melodía al ESP32 a través de la comunicación
serial2
    Enviar un código de estado 200 junto con el mensaje "Reproduciendo
melodía: [nombre de la melodía]" como respuesta al cliente

// Manejar la solicitud del icono de la página web
```

```
Función handleIcon():
    Abrir el archivo "icono.jpg" desde SPIFFS
    Si no se puede abrir el archivo:
        Enviar un código de estado 404 junto con el mensaje "Icono no
encontrado" como respuesta al cliente
        Retornar

    Enviar el contenido del archivo de imagen al cliente con el tipo MIME
"image/jpg"
    Cerrar el archivo

// Manejar la solicitud de la imagen de la página web
Función handleImage():
    Abrir el archivo "image.png" desde SPIFFS
    Si no se puede abrir el archivo:
        Enviar un código de estado 404 junto con el mensaje "Imagen no
encontrada" como respuesta al cliente
        Retornar

    Enviar el contenido del archivo de imagen al cliente con el tipo MIME
"image/png"
    Cerrar el archivo

// Bucle principal
Función loop():
    Manejar las solicitudes del cliente al servidor
    Si hay datos disponibles en Serial2:
        Leer el mensaje recibido hasta el carácter de nueva línea
        Si el mensaje comienza con "M:" (indicando una melodía recibida):
            Extraer el nombre de la melodía del mensaje
            Eliminar los espacios en blanco al principio y al final del
nombre de la melodía
            Si la cantidad de melodías creadas es menor que MAX_MELODIES:
                Almacenar el nombre de la melodía en el array de melodías
creadas
                Incrementar el contador de melodías creadas
                Mostrar "Melodía recibida: [nombre de la melodía]" en la
consola serial

// Fin del pseudocódigo.
```

4. CONCLUSIONES

Tras completar el proyecto de desarrollo del sistema de creación y reproducción de melodías utilizando el Arduino Mega 2560 y la tarjeta ESP32-DevKitV1, se han obtenido varias conclusiones significativas que destacan tanto los logros como los desafíos superados durante el proceso.

Esta experiencia ha permitido profundizar en la programación de sistemas embebidos y la integración de comunicaciones cableadas e inalámbricas mediante Arduino. Desde el diseño de la interfaz de usuario hasta la implementación de la lógica para la creación y reproducción de melodías, se ha explorado diversos aspectos de la programación en entornos embebidos, adquiriendo un entendimiento más completo de estas tecnologías.

En cuanto a la implementación de la comunicación entre el Arduino y la ESP32, inicialmente se presentaron problemas en la transmisión y recepción de nombres de melodías. La solución fue inicializar un nuevo puerto serial: '`Serial1`', para el código fuente de las melodías, y '`Serial2`', para el código fuente del WiFi, lo que permitió establecer una comunicación adecuada y asegurando que los nombres de las melodías fueran correctamente enviados y recibidos.

Respecto a la reproducción de las melodías seleccionadas por el usuario, se encontró que las melodías se reproducían de manera duplicada debido a que el nombre de la melodía se enviaba dos veces. La solución fue definir una variable de control, '`melodyPlaying`', para gestionar la reproducción, asegurando que las melodías se reprodujeran una sola vez, evitando duplicaciones.

La implementación del sistema de archivos SPIFFS en la ESP32 para gestionar los recursos web, como archivos HTML, JPG y PNG, mejoró la organización y flexibilidad del código.

Durante el desarrollo del proyecto, se enfrentaron diversos desafíos técnicos, como la gestión de la visualización en el LCD y la precisión en la detección de las pulsaciones del codificador rotativo. Se implementaron soluciones para limpiar adecuadamente el área de visualización del LCD y se añadieron retardos para evitar rebotes en el codificador rotatorio, mejorando la precisión en la interacción del usuario.

La documentación adecuada del proceso de desarrollo, incluyendo esquemas de montaje, algoritmos utilizados y vídeos demostrativos del funcionamiento del sistema, fue esencial para la replicación del proyecto y su comprensión por parte de otros desarrolladores.

En conclusión, el proyecto ha sido un éxito, logrando la implementación de su sistema robusto y funcional para la creación y reproducción de melodías mediante comunicación WiFi. Los conocimientos adquiridos y las habilidades desarrolladas

durante este proyecto son valiosos para futuros desarrollos en el ámbito de los sistemas embebidos y el Internet de las Cosas (IoT), demostrando importancia de la integración entre teoría y práctica en el desarrollo de soluciones efectivas.

5. ANEXO

Como complemento a esta memoria, adjuntamos los códigos *melodies.ino* y *wifi.ino*, los cuales se encuentran debidamente comentados para una mejor comprensión y referencia. También incluimos el código HTML que se ha realizado.

melodies.ino:

```
#include <rgb_lcd.h>
#include "pitches.h"

#define MELODY_NAME_LEN 15
#define MELODY_NOTES 30 // Núm. máx. de notas por melodía
#define MAX_MELODIES 10 // Núm. máx. de melodías almacenables

// Pines del rotary
int pinA = 2; // CLK
int pinB = 3; // DT
int pinSw = 4; // Switch

// Pin del zumbador
int pinZ = 8;

// Variables globales
int currentNoteIndex = 0;
int currentMelodyIndex = 0;
int noteSelection = 0;
int melodyCount = 0;
bool inCreationMode = true;
bool confirmSave = false;
unsigned long lastSwitchPressTime = 0;
int switchPressCount = 0;
unsigned long debounceTime = 200;

const int noteFrequencies[] = { NOTE_C4, NOTE_D4, NOTE_E4, NOTE_F4,
NOTE_G4, NOTE_A4, NOTE_B4, NOTE_C5 };
const char* noteNames[] = { "DO", "RE", "MI", "FA", "SOL", "LA", "SI",
"DO" };

rgb_lcd lcd;

typedef struct {
```

```
    int frequency;
    int duration;
} Note;

typedef struct {
    char name[MELODY_NAME_LEN + 1];
    Note notes[MELODY_NOTES];
    int noteCount;
} Melody;

// Dónde se almacenan las melodías creadas
Melody melodies[MAX_MELODIES];

// Melodías predefinidas
const Melody predefinedMelodies[] = {
    // Star Wars
    { "Melodia A",
        {
            {NOTE_G4, 500}, {NOTE_G4, 500}, {NOTE_G4, 500}, {NOTE_E4,
375}, {NOTE_B4, 125}, {NOTE_G4, 500}, {NOTE_E4, 375}, {NOTE_B4, 125},
{NOTE_G4, 1000}, // Primer compás
            {NOTE_D5, 500}, {NOTE_D5, 500}, {NOTE_D5, 500}, {NOTE_E5,
375}, {NOTE_B4, 125}, {NOTE_G4, 500}, {NOTE_E4, 375}, {NOTE_B4, 125},
{NOTE_G4, 1000}, // Segundo compás
            {-1, 0}
        }
    },
    // Harry Potter
    { "Melodia B",
        {
            {NOTE_E5, 300}, {NOTE_G5, 300}, {NOTE_FS5, 300}, {NOTE_E5,
300}, {NOTE_B5, 300}, {NOTE_A5, 300}, {NOTE_FS5, 300}, {NOTE_D5, 300},
{NOTE_B5, 300}, // Primer compás
            {NOTE_A5, 300}, {NOTE_FS5, 300}, {NOTE_E5, 300}, {NOTE_G5,
300}, {NOTE_FS5, 300}, {NOTE_D5, 300}, {NOTE_A5, 300}, {NOTE_FS5, 300},
{NOTE_E5, 600}, // Segundo compás
            {-1, 0}
        }
    },
    // Indiana Jones
    { "Melodia C",
        {
            {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_G4, 600}, {NOTE_C4,
300}, {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_D4, 300}, {NOTE_E4, 600}, //
Primer compás
            {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_G4, 600}, {NOTE_C4,
300}, {NOTE_E4, 300}, {NOTE_F4, 300}, {NOTE_D4, 300}, {NOTE_E4, 600}, //
Segundo compás
            {-1, 0}
```

```
        }
    },
    // Piratas del Caribe
    { "Melodia D",
        {
            {NOTE_E4, 300}, {NOTE_G4, 300}, {NOTE_A4, 300}, {NOTE_B4,
300}, {NOTE_C5, 300}, {NOTE_D5, 300}, {NOTE_E5, 300}, {NOTE_G5, 600}, //
Primer compás
            {NOTE_E5, 300}, {NOTE_D5, 300}, {NOTE_C5, 300}, {NOTE_B4,
300}, {NOTE_A4, 300}, {NOTE_G4, 300}, {NOTE_E4, 300}, {NOTE_D4, 600}, //
Segundo compás
            {NOTE_E4, 300}, {NOTE_G4, 300}, {NOTE_A4, 300}, {NOTE_B4,
300}, {NOTE_C5, 300}, {NOTE_D5, 300}, {NOTE_E5, 300}, {NOTE_G5, 600}, //
Tercer compás
            {-1, 0}
        }
    }
};

// Configuración inicial del sistema
void setup() {
    Serial.begin(115200);
    Serial1.begin(115200);
    setupPins();
    setupLCD();
}

// Configuración de los pines del Arduino
void setupPins() {
    pinMode(pinA, INPUT);
    pinMode(pinB, INPUT);
    pinMode(pinSw, INPUT_PULLUP);
    pinMode(pinZ, OUTPUT);
}

// Configuración del LCD
void setupLCD() {
    lcd.begin(16, 2);
    lcd.setRGB(255, 255, 255);
    lcd.print("Creador de");
    lcd.setCursor(0, 2);
    lcd.print("melodias");
    delay(2000);
    lcd.clear();
    lcd.print("Introduce 10");
    lcd.setCursor(0, 2);
    lcd.print("notas");
    delay(2000);
    lcd.clear();
```

```
    lcd.print("Nota: DO");
}

// Maneja la rotación del encoder y actualiza la nota seleccionada
void handleEncoder() {
    static int pinALastState = digitalRead(pinA);
    int pinAState = digitalRead(pinA);

    if (pinAState != pinALastState && pinAState == HIGH) { // Si hay un
cambio de estado
        if (digitalRead(pinB) == LOW) {
            noteSelection = (noteSelection + 1) % 8; // Cambia la nota
hacia delante
        } else {
            noteSelection = (noteSelection - 1 + 8) % 8; // Cambia la
nota hacia atrás
        }
        lcd.setCursor(6, 0);
        lcd.print("    "); // Limpia el área donde se muestra la nota
        lcd.setCursor(6, 0);
        lcd.print(noteNames[noteSelection]);
    }
    pinALastState = pinAState;
}

// Maneja las pulsaciones del switch y llama a las funciones apropiadas
según el número de pulsaciones
void handleSwitchPress() {
    bool switchPressed = (digitalRead(pinSw) == LOW);

    if (switchPressed && (millis() - lastSwitchPressTime) > debounceTime)
{
        switchPressCount++;
        lastSwitchPressTime = millis();
        delay(100); // Se aplica un pequeño retraso para evitar rebotes
adicionales
    }

    if (!switchPressed && (millis() - lastSwitchPressTime) >
debounceTime) {
        if (switchPressCount == 1) { // 1 pulsación
            addNote();
        } else if (switchPressCount == 2) { // 2 pulsaciones
            saveMelody();
        } else if (switchPressCount == 3) { // 3 pulsaciones
            deleteLastNote();
        } else if (switchPressCount == 4) { // 4 pulsaciones
            deleteAllNotes();
        }
```

```
        switchPressCount = 0; // Reinicia el contador de pulsaciones
    }
}

// Añade la nota seleccionada a la melodía actual
void addNote() {
    melodies[currentMelodyIndex].notes[currentNoteIndex] = {
noteFrequencies[noteSelection], 250 };
    currentNoteIndex++;
    lcd.setCursor(0, 1);
    lcd.print("Notas: ");
    lcd.print(currentNoteIndex); // Muestra el núm. de notas actuales
    lcd.print("  "); // Limpia el área para evitar errores en la
visualización
}

// Guarda la melodía actual, gestiona la confirmación para crear otra
melodía y envía la melodía creada
void saveMelody() {
    melodies[currentMelodyIndex].noteCount = currentNoteIndex; //
Establece el número de notas en la melodía
    melodyCount++;
    snprintf(melodies[currentMelodyIndex].name, MELODY_NAME_LEN,
"melodia%d", melodyCount); // Asigna un nombre a la melodía
    lcd.clear();
    lcd.print("Melodia guardada");

    // Variable para guardar el nombre de la melodía creada
    const char* melodyName = melodies[currentMelodyIndex].name;

    delay(2000);
    lcd.clear();
    lcd.print("Crear otra?");
    lcd.setCursor(0, 1);
    lcd.print("[Si] No");
    confirmSave = true;
    confirmSaveLoop();

    // Se envía el nombre de la melodía creada actualmente a la ESP32
    sendMelodyToESP32(melodyName);
}

// Maneja el proceso de confirmación para crear una nueva melodía
void confirmSaveLoop() {
    static int pinALastState = digitalRead(pinA);
    int pinAState;

    while (true) {
        pinAState = digitalRead(pinA);
```

```
        if (pinAState != pinALastState && pinAState == HIGH) {
            if (digitalRead(pinB) == LOW) {
                confirmSave = !confirmSave;
            } else {
                confirmSave = !confirmSave;
            }
            lcd.setCursor(0, 1);
            lcd.print("        "); // Limpia el área donde se muestra la
selección
            lcd.setCursor(0, 1);
            lcd.print(confirmSave ? "[Si] No" : " Si [No]");
        }
        pinALastState = pinAState;

        if (digitalRead(pinSw) == LOW) {
            delay(200);
            if (confirmSave) {
                currentNoteIndex = 0;
                currentMelodyIndex = melodyCount;
                lcd.clear();
                lcd.print("Nota: DO");
            } else {
                lcd.clear();
                lcd.print("Total melodias:");
                lcd.setCursor(0, 1);
                lcd.print(melodyCount);
                while (true){}; // Mantiene el mensaje indefinidamente
            }
            break;
        }
    }
}

//  Elimina la última nota añadida
void deleteLastNote() {
    if (currentNoteIndex > 0) { // Si hay notas para eliminar
        currentNoteIndex--; // Decrementa el índice de notas
        lcd.setCursor(0, 1);
        lcd.print("Notas: ");
        lcd.print(currentNoteIndex); // Muestra el núm. de notas actuales
        lcd.print("  "); // Limpia el área para evitar errores en la
visualización
}

// Elimina todas las notas de la melodía actual
void deleteAllNotes() {
    currentNoteIndex = 0; // Reinicia el índice de notas
    lcd.setCursor(0, 1);
    lcd.print("Notas: ");
```

```
    lcd.print(currentNoteIndex); // Muestra el núm. de notas actuales = 0
    lcd.print("  "); // Limpia el área para evitar errores en la
visualización
}

// Reproduce las notas
void playMelody(int frequency, int duration) {
    if (frequency == 0) {
        noTone(pinZ);
    } else {
        tone(pinZ, frequency, duration);
    }
    delay(duration);
}

// Reproduce la melodía seleccionada
void playMelodiesFromESP32(String melodyName) {

    for (int i = 0; i < 4; i++) {
        if (String(predefinedMelodies[i].name) == melodyName) {
            for (int j = 0; j < 50 &&
predefinedMelodies[i].notes[j].frequency != -1; j++) {
                playMelody(predefinedMelodies[i].notes[j].frequency,
predefinedMelodies[i].notes[j].duration); // Reproduce cada nota de la
melodía predefinida
            }
            return;
        }
    }
    for (int i = 0; i < (melodyCount); i++) {
        if (String(melodies[i].name) == melodyName) {
            for (int j = 0; j < melodies[i].noteCount; j++) {
                playMelody(melodies[i].notes[j].frequency,
melodies[i].notes[j].duration); // Reproduce cada nota de la melodía
creada
            }
            return;
        }
    }

    Serial.println("Error: Melodía no encontrada");
}

// Maneja el envío de las melodías creadas
void sendMelodyToESP32(const char* melodyName) {
    Serial1.print("M:"); // Envía el prefijo para identificar el mensaje
    Serial1.println(melodyName); // Envía el nombre de la melodía
    Serial.print("Sending melody: ");
```

```
    Serial.println(melodyName); // Muestra en el serial el nombre de la
melodía enviada
    delay(100); // Espera un pequeño retraso para asegurar un envío
correcto
}

// Bucle principal del programa
void loop() {
    handleEncoder();
    handleSwitchPress();

    if (Serial1.available() > 0) { // Si hay datos disponibles en el
puerto serial1
        String message = Serial1.readStringUntil('\n'); // Lee el mensaje
recibido hasta el carácter de nueva línea
        if (message.startsWith("M:")) { // Si el mensaje empieza con
"M:""
            String melodyName = message.substring(2); // Extrae el nombre
de la melodía del mensaje
            melodyName.trim(); // Elimina espacios en blanco al principio
y al final
            playMelodiesFromESP32(melodyName); // Reproduce la melodía
recibida
        }
    }
}
```

wifi.ino:

```
#include <WiFi.h>
#include <WebServer.h>
#include "FS.h"
#include "SPIFFS.h"

// Configuración de las credenciales WiFi
const char* ssid = "ESP32-AP04";
const char* password = "123456789";

// Creación de un servidor web en el puerto 80
WebServer server(80);

struct Melody {
    String name;
};

#define MAX_MELODIES 10
Melody createdMelodies[MAX_MELODIES];
int createdMelodyCount = 0;
```

```
bool melodyPlaying = false; // Variable de control para evitar múltiples
reproducciones simultáneas

// Configuración inicial del sistema
void setup() {
    Serial.begin(115200);
    Serial2.begin(115200);
    setupSPIFFS();
    setupWiFi();
    setupServer();
}

// Inicializa el sistema de archivos SPIFFS
void setupSPIFFS() {
  if (!SPIFFS.begin(true)) {
        Serial.println("SPIFFS Mount Failed");
        return;
  }
}

// Configura el punto de acceso WiFi
void setupWiFi() {
    Serial.println();
    Serial.println("Configuring access point...");
    WiFi.softAP(ssid, password);
    IPAddress myIP = WiFi.softAPIP();
    Serial.print("AP IP address: ");
    Serial.println(myIP);
}

// Configura las rutas y funciones del servidor web
void setupServer() {
    server.on("/", handleRoot); // Maneja la ruta raíz "/"
    server.on("/play", handlePlay); // Maneja la ruta "/play"
    server.on("/melodies", handleMelodies); // Maneja la ruta "/melodies"
    server.on("/icono.jpg", handleIcon); // Maneja la ruta "/icono.jpg"
    server.on("/image.jpg", handleImage); // Maneja la ruta "/image.jpg"
    server.begin();
    Serial.println("HTTP server started");
}

// Maneja la solicitud de la página HTML principal
void handleRoot() {
    File file = SPIFFS.open("/index.html"); // Abre el archivo index.html
desde SPIFFS
    if (!file) {
        server.send(500, "text/plain", "Failed to open /index.html"); //
Envía un error si el archivo no se puede abrir
        return;
```

```
    }

    String html;
    while (file.available()) {
        html += char(file.read()); // Lee el contenido del archivo
    }
    file.close();

    server.send(200, "text/html", html); // Envía el contenido HTML al
cliente
}

// Maneja la solicitud del listado de melodías
void handleMelodies() {
    String melodiesList = "<b>Melodías predefinidas</b>";
    for (int i = 0; i < 4; i++) {
        melodiesList += "<li><a href=\"#\" onclick=\"playMelody('Melodia
" + String((char)('A' + i)) + "')\">Melodia " + String((char)('A' + i)) +
"</a></li>";
    }

    melodiesList += "<hr>";

    melodiesList += "<b>Melodías creadas</b>";
    for (int i = 0; i < createdMelodyCount; i++) {
        melodiesList += "<li><a href=\"#\" onclick=\"playMelody('" +
createdMelodies[i].name + "')\">" + createdMelodies[i].name +
"</a></li>";
    }

    server.send(200, "text/plain", melodiesList); // Envía el listado de
melodías al cliente
}

// Maneja la solicitud de reproducción de las melodías
void handlePlay() {
    if (!server.hasArg("melody") || melodyPlaying) {
        server.send(400, "text/plain", "Invalid request");
        return;
    }

    String melody = server.arg("melody"); // Obtiene el nombre de la
melodía
    melody.trim();

    Serial.println("M:" + melody);
    Serial2.println("M:" + melody);

    server.send(200, "text/plain", "Melody playing: " + melody);
```

```
    melodyPlaying = true; // Marca que una melodía está en reproducción
    delay(1000); // Espera de corta duración para evitar múltiples
reproducciones simultáneas
    melodyPlaying = false; // Marca que la reproducción ha terminado
}

// Maneja la solicitud del icono de la página web
void handleIcon() {
    File file = SPIFFS.open("/icono.jpg"); // Abre el archivo del icono
desde SPIFFS
    if (!file) {
        server.send(404, "text/plain", "Icon not found");
        return;
    }

    server.streamFile(file, "image/jpg"); // Envía el archivo de imagen
al cliente
    file.close();
}

// Maneja la solicitud de la imagen de la página web
void handleImage() {
    File file = SPIFFS.open("/image.png"); // Abre el archivo de imagen
desde SPIFFS
    if (!file) {
        server.send(404, "text/plain", "Image not found");
        return;
    }

    server.streamFile(file, "image/png"); // Envía el archivo de imagen
al cliente
    file.close();
}

void loop() {
    server.handleClient(); // Maneja las solicitudes del cliente

    // Lee los mensajes de Serial2
    if (Serial2.available()) {
        String message = Serial2.readStringUntil('\n'); // Lee un mensaje
hasta el carácter de nueva línea
        if (message.startsWith("M:")) {
            String melodyName = message.substring(2); // Extrae el nombre
de la melodía del mensaje
            melodyName.trim(); // Elimina espacios y nuevas líneas
sobrantes

            if (createdMelodyCount < MAX_MELODIES) {
```

```
                createdMelodies[createdMelodyCount].name = melodyName; //
Almacena el nombre de la melodía en el array
                createdMelodyCount++;
                Serial.println("Melody received: " + melodyName);
            }
        }
    }
}
```

código HTML de index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Melodies</title>
    <!-- Icono de la página -->
    <link rel="icon" href="icono.jpg" type="image/jpg">
    <script>
        // Función para enviar una solicitud para reproducir una melodía
        function playMelody(melody) {
            var xhr = new XMLHttpRequest();
            xhr.open('GET', '/play?melody=' + melody, true); // Solicitud
GET al servidor con el nombre de la melodía
            xhr.send(); // Envía la solicitud
        }

        // Función para cargar la lista de melodías desde el servidor
        function loadMelodies() {
            var xhr = new XMLHttpRequest();
            xhr.open('GET', '/melodies', true); // Solicitud GET al
servidor para obtener la lista de melodías
            xhr.onload = function() {
                if (xhr.status === 200) {
                    document.getElementById('melodies').innerHTML =
xhr.responseText; // Actualiza el contenido de la lista de melodías
                } else {
                    console.error('Failed to load melodies');
                }
            };
            xhr.send(); // Envía la solicitud
        }

        // Función para actualizar la lista de melodías periódicamente
        function refreshMelodies() {
            loadMelodies(); // Carga la lista de melodías
            setTimeout(refreshMelodies, 5000); // Actualiza cada 5
segundos
        }
```

```
        // Inicializa las funciones al cargar la página
        window.onload = function() {
            loadMelodies(); // Carga la lista de melodías al iniciar
            refreshMelodies(); // Inicia la actualización periódica
        };
    </script>
</head>
<body>
    <h1>Melodias disponibles</h1>
    <div style="background-color: gainsboro;"> <!-- Contiene la lista de
melodías disponibles, que se cargará dinámicamente desde el servidor -->
        <ul id="melodies">
            <!-- La lista de melodías se cargará aquí -->
        </ul>
    </div>
    <img src="image.png" alt="page image" style="display: block; margin-
top: 50px;">
</body>
</html>
```